

UNIVERSIDAD DON BOSCO

FACULTAD DE INGENIERÍA

Desarrollo de Software para Móviles

Investigación sobre Jetpack Compose aplicada al proyecto SuperApp

Grupo teórico No. 01T

Docente:

Ing. Alexander Alberto Siguenza Campos

Alumnos:

Barrera Guzmán Helen Tatiana BG240719

Barahona García Miguel Ángel BG191322

Merino Hernández Gabriela Guadalupe MH203046

Méndez Díaz Neris Moisés MD161918

.

Resumen

El presente trabajo investiga Jetpack Compose y analiza su aplicación en el proyecto Super App, una solución diseñada para el control de presupuesto y la elaboración de listas de compras en un supermercado. El primer avance entregado por el equipo se enfoca en módulos funcionales ejecutados en consola, incluyendo perfiles de administrador y cliente, registro de usuarios, gestión de productos, promociones, estadísticas, presupuesto y listado de compras. A partir de este estado actual del proyecto, se estudia como convertir las interacciones de consola en una interfaz Android desarrollada con Kotlin y Jetpack Compose. Se explican conceptos como el enfoque declarativo, las funciones composables, el estado, la recomposición, los componentes de interfaz, la navegación y la separación entre UI y lógica de negocio. Además, se propone una correspondencia entre los módulos actuales y las futuras pantallas móviles, identificando que responsabilidades deberían permanecer en servicios o capas de datos y cuales deberían ser manejadas por Compose. La investigación concluye que Jetpack Compose es una alternativa adecuada para la siguiente etapa, siempre que el equipo migre el proyecto a una configuración completa de Android y mantenga una separación clara entre presentación, estado, lógica y datos.

Palabras clave: Jetpack Compose, Kotlin, Android, SuperApp, **presupuesto**, supermercado, **lista** de compras, **interfaz declarativa**.

1. Introducción

El desarrollo de aplicaciones móviles implica elegir tecnologías que permiten crear interfaces efectivas, fáciles de mantener y capaces de reaccionar a las acciones del usuario. En Android, Jetpack Compose es el conjunto moderno de herramientas para diseñar interfaces, y se basa en un enfoque declarativo que describe la interfaz mediante código Kotlin. Según Android Developers, este método permite representar la interfaz según el estado actual de la aplicación y actualizarla automáticamente cuando ese estado cambia (Android Developers, 2026a, 2026b).

El proyecto SuperApp se enfoca en resolver un problema específico: ayudar al usuario a planificar compras y controlar su presupuesto. Según el README del primer avance, el administrador puede gestionar usuarios, productos, promociones y estadísticas, mientras que los clientes pueden administrar su presupuesto y crear listas de compras.

En la actualidad, el proyecto no es todavía una aplicación Android completa. La configuración inicial utiliza Kotlin/JVM y el plugin de aplicación, mientras que el módulo dsm incluye solo una configuración Android parcial. Esta situación hace de Jetpack Compose una tecnología de migración y evolución del proyecto: la lógica funcional ya definida debe conservarse, y la capa de interacción debe trasladarse gradualmente a pantallas móviles.

El objetivo de esta investigación es estudiar Jetpack Compose aplicándolo al proyecto SuperApp, relacionando sus conceptos con las funcionalidades reales propuestas. Por lo tanto, el documento no presenta componentes de Compose como si ya

estuvieran implementados, sino como una propuesta tecnológica para la siguiente fase del desarrollo.

2. Descripción del proyecto SuperApp

Propósito del proyecto

El README del primer avance presenta a SuperApp como una aplicación para el control de presupuesto en un supermercado. Su objetivo principal es ayudar a los usuarios a gestionar sus compras y su presupuesto, mientras que el perfil administrativo puede consultar y administrar elementos generales del sistema. Por lo tanto, la propuesta busca una solución que integre un catálogo de productos, compras planificadas y un control financiero básico.

Roles identificados

En cuanto a los roles identificados, el proyecto establece dos perfiles principales: administrador y cliente. El README ofrece credenciales de prueba para acceder como administrador, y para utilizar el módulo de clientes, se debe primero registrar un nuevo usuario. Esta diferenciación de responsabilidades es importante para el diseño futuro de la interfaz, ya que no todas las pantallas deberían estar disponibles para ambos perfiles.

Funcionalidades del primer avance.

Tabla 1

Funciones identificadas en el primer avance

Rol	Funcionalidad indicada en el README
Administrador	Gestión de usuarios.
Administrador	Gestión de productos.
Administrador	Gestión de promociones.
Administrador	Consulta de estadísticas.
Cliente	Gestión del presupuesto.
Cliente	Elaboración del listado de compras.
Cliente	Registro de usuario para explorar el módulo de cliente

3. ¿Qué es Jetpack Compose?

Jetpack Compose es el conjunto de herramientas moderno de Android para crear interfaces de usuario. Su característica principal es el enfoque declarativo: en lugar de cambiar

manualmente una estructura de vistas para reflejar cada actualización, el desarrollador describe la interfaz basándose en los datos y el estado actual. Cuando cambia el estado, Compose vuelve a ejecutar los elementos afectados y actualiza la interfaz automáticamente.

Una función composable es una función de Kotlin que forma parte de la construcción de la interfaz. Estas funciones permiten dividir una pantalla en componentes pequeños y reutilizables. Por ejemplo, en una aplicación como SuperApp, un producto podría representarse con un componente reutilizable que reciba como parámetros el nombre, el precio, la imagen, la categoría y la cantidad. La forma declarativa de Compose es especialmente útil cuando la información cambia con frecuencia.

En SuperApp, el total de una lista de compras puede depender de varios productos y cantidades. Si el usuario modifica una cantidad, el estado cambia y la interfaz se actualiza automáticamente para mostrar el nuevo subtotal y el nuevo total, sin necesidad de actualizar manualmente cada Text o View.

Composición y recomposición

Android Developers explica que la composición se produce al ejecutar inicialmente los composables para describir la UI, mientras que la recomposición consiste en volver a ejecutar los composables que pueden haber cambiado debido a una modificación del estado. En SuperApp, esto puede utilizarse para reflejar cambios en productos seleccionados, cantidades, presupuesto disponible o resultados de estadísticas.

Estado en Compose

El estado es cualquier valor que puede cambiar con el tiempo y que influye en lo que el usuario ve. Compose ofrece APIs como remember y mutableStateOf para guardar y seguir ciertos valores. Cuando cambia un valor de estado que un composable está leyendo, Compose programa la recomposición de las partes que dependen de ese valor.

4. Componentes de Jetpack Compose aplicables a SuperApp

La siguiente selección representa componentes que podrían utilizarse en la etapa Android. No se presentan como componentes actualmente existentes en el primer avance, sino como elementos propuestos para la interfaz móvil.

Tabla 2

Componentes de Jetpack Compose aplicables a SuperApp

Componente	Aplicación propuesta en SuperApp
Text	Nombres de productos, precios, mensajes, títulos y resultados.
TextField / OutlinedTextField	Correo, contraseña, búsqueda, presupuesto y otros datos introducidos por el usuario.
Button	Registrar, iniciar sesión, agregar producto, guardar presupuesto y confirmar acciones.
Card	Presentación agrupada de productos, promociones o resúmenes.
Column / Row	Organización vertical u horizontal de información y controles.
LazyColumn	Listados de productos y elementos de la lista de compras.
Scaffold	Estructura general de pantallas con barra superior, contenido y navegación.

Componente	Aplicación propuesta en SuperApp
NavigationBar	Acceso rápido a destinos principales del cliente.

5. Transformación de los módulos actuales a una interfaz Compose

La migración no debe entenderse como reemplazar todo el proyecto. La idea es conservar la funcionalidad de negocio que ya fue definida y sustituir progresivamente la interacción por consola por una interfaz gráfica móvil. El siguiente mapeo constituye una propuesta de diseño basada en los módulos y funciones descritos en el README.

Tabla 3

Transformación de los módulos a una interfaz Compose

Primer avance	Propuesta Android + Compose	Responsabilidad
Registro de usuario	RegisterScreen	Formulario de registro y validación.
Inicio de sesión	LoginScreen	Ingreso y selección del flujo según rol.
Gestión de productos	ProductsScreen / ProductFormScreen	Consultar y administrar productos según rol.
Promociones	PromotionsScreen	Definir presupuesto y visualizar disponible.
Listado de compras	ShoppingListScreen	Agregar, modificar y eliminar productos.
Estadísticas	StatisticsScreen	Presentar información resumida y gráficos.
Menús de consola	NavHost / NavigationBar	Navegar entre las funciones de la aplicación.

6. Aplicación del estado y recomposición al presupuesto y lista de compras.

El módulo de presupuesto es un excelente ejemplo para mostrar cómo se utiliza Compose. El usuario puede establecer un monto disponible y luego añadir productos al listado de compras. El total estimado cambia cuando se modifica la cantidad o se agregan o eliminan productos. En una interfaz declarativa, esos valores pueden mostrarse como estado de la pantalla.

Un posible estado de la pantalla podría incluir un presupuesto, una lista de productos seleccionados y un total calculado. La interfaz gráfica recibiría ese estado y mostraría el

presupuesto, el total gastado y lo que queda disponible. Cuando el usuario realiza una acción, como aumentar la cantidad de un producto, la lógica actualiza el estado y Compose vuelve a mostrar la información afectada.

Android sugiere que la interfaz gráfica debe recibir el estado y exponer eventos, en lugar de modificar directamente el estado de la aplicación. Este flujo unidireccional ayuda a reutilizar componentes, facilita las pruebas y permite controlar mejor dónde ocurren los cambios.

7. Navegación propuesta para SuperApp

La navegación es necesaria porque el proyecto incluye varias funciones independientes. Navigation Compose permite moverse entre destinos compuestos y usar un NavController para gestionar la ruta de la aplicación.

Para el cliente, una estructura posible sería: Inicio → Productos → Lista de compras → Presupuesto → Estadísticas.

Para el administrador, el flujo podría incluir: Inicio administrativo → Usuarios → Productos → Promociones → Estadísticas. Cada destino debe estar disponible solo si el usuario tiene el rol correspondiente, por lo que un cliente no podrá acceder a las funciones administrativas.

Una barra de navegación inferior es adecuada cuando hay entre tres y cinco destinos de igual importancia, mientras que un menú lateral puede servir como alternativa si hay más secciones administrativas.

Android Developers recomienda usar la NavigationBar principalmente para cambiar entre destinos principales en pantallas pequeñas y cuando haya entre tres y cinco destinos de importancia similar.

8. Separación entre interfaz y lógica de negocio.

La migración a Compose no implica meter toda la lógica dentro de las pantallas. La documentación de arquitectura de Android sugiere que los eventos de la interfaz de usuario deben ser procesados por una capa de estado, como un ViewModel, y que la interfaz observe ese estado. Este enfoque está relacionado con el flujo de datos unidireccional.

Para SuperApp, una posible estructura futura sería: modelos → servicios/repositorios → ViewModels → pantallas de Compose.

Los modelos representan entidades como usuarios, productos y promociones; los servicios o repositorios se encargan de gestionar los datos; los ViewModels manejan el estado y los eventos de cada pantalla; y los composables se centran en mostrar la información y en emitir acciones del usuario.

Esta propuesta es muy importante ya que el primer paso ya separa conceptualmente responsabilidades relacionadas con autenticación, productos, listas, promociones y estadísticas. La migración debe mantener esa separación en lugar de mover todas las operaciones a las funciones composable.

9. Ventajas de utilizar Jetpack Compose en SuperApp

- • Integración con Kotlin. La interfaz se construye en el mismo lenguaje utilizado por el proyecto.
- • Modelo declarativo. Resulta adecuado para representar cambios frecuentes en
- presupuesto, cantidades y listas.
- • Componentización. Productos, tarjetas, botones y resúmenes pueden
- convertirse en composables reutilizables.
- • Navegación moderna. Navigation Compose permite organizar las diferentes

- pantallas.
- • Adaptabilidad. Compose facilita la creación de interfaces capaces de adaptarse a distintos tamaños de pantalla.
- • Separación de estado y UI. Su integración con ViewModel y flujo de datos unidireccional favorece una arquitectura mantenible

10. **Relación con los requisitos del proyecto.**

Jetpack Compose puede ayudar principalmente a cumplir los requisitos relacionados con la interfaz y la navegación. Las funciones ya definidas, como roles, productos, promociones, presupuesto, lista de compras y estadísticas, pueden convertirse en módulos que sean visibles y accesibles a través de la navegación. Sin embargo, Compose en sí mismo no maneja la persistencia de datos, la autenticación, la generación de gráficos o la implementación de reglas de negocio. Para estas tareas es necesario combinarlo con otras tecnologías y capas que se encarguen de cada responsabilidad específica.

Además, el uso de Compose puede facilitar la creación de una interfaz consistente para diferentes roles.Por ejemplo, un mismo componente para mostrar productos puede reutilizarse tanto para un cliente que agrega un artículo a su lista como para un administrador que consulta o modifica información del catálogo, adaptándose las acciones según el rol y el contexto en el que se use.

11. **Conclusiones**

La investigación mostró que Jetpack Compose es una buena opción para seguir desarrollando el proyecto SuperApp como una aplicación para Android. Sin embargo, actualmente la aplicación aún está en una fase inicial donde el código principal sigue siendo una implementación de consola y utiliza Kotlin/JVM como base.Por lo tanto, Compose será parte de una etapa posterior del desarrollo.

Una de las grandes ventajas de usar Compose en SuperApp radica en su modelo declarativo y en cómo conecta el estado con la interfaz. Esto es especialmente útil en funcionalidades como el presupuesto y la lista de compras, donde los datos cambian frecuentemente, y la recomposición automática ayuda a mantener la interfaz actualizada.

Además, las funcionalidades relacionadas con administrador y cliente permiten crear una navegación específica. Con Navigation Compose, los menús actuales podrían convertirse en pantallas móviles, mientras que componentes como LazyColumn, Card, TextField, Button y NavigationBar servirían como elementos visuales básicos para los módulos.

Para la migración, es importante mantener la separación de responsabilidades. Los composables deberán encargarse principalmente de mostrar la interfaz y generar eventos, mientras que la lógica y los datos seguirán en capas adecuadas. De este modo, el equipo podrá aprovechar lo que ya se ha hecho y transformarlo gradualmente en una aplicación móvil mantenible.

Referencias

Android Developers. (2026a). *State and Jetpack Compose.* https://developer.android.com/develop/ui/compose/state

Android Developers. (2026b). *Guide to app architecture.* https://developer.android.com/topic/architecture

Gabriela110. (2026). *Proyecto—Primer avance [Código fuente]. GitHub.* https://github.com/Gabriela110/Proyecto---Primer-Avance-